kyriosMICA

Institut de Recherche en Bio-Informatique Systémique Quantique

GUIDE TECHNIQUE DE DÉCHIFFREMENT

Protocole MICA-OTP — Manuel Complet pour Cryptanalystes

Référence	KM-DOC-007-2026
Version	1.0
Auteur	Cyrille Egnon DAVOH — Fondateur & Chercheur Principal
Institution	kyriosMICA — Bio-Informatique Systémique Quantique
Date	Mars 2026 — Bénin, Afrique de l'Ouest
Classification	Document Technique Public
Destinataires	Cryptanalystes, Chercheurs, Institutions

Decoding Life. Encoding the Future.

kyriosmica.com · direction@kyriosmica.com

© 2026 kyriosMICA · Cyrille Egnon DAVOH · Tous droits réservés · direction@kyriosmica.com

Objet

RÉSUMÉ

Ce guide fournit toutes les informations nécessaires pour comprendre, implémenter et tenter de déchiffrer un message chiffré avec le protocole MICA-OTP du système Qudits-36 de kyriosMICA. Il est remis à tout participant au Défi Cryptographique Mondial.

1. Vue d'Ensemble

1.1 MICA-OTP vs OTP Binaire Classique

Propriété	OTP Binaire Classique	MICA-OTP Ternaire
Alphabet	{0, 1}	$T_3 = \{-1, 0, +1\}$
Opération	XOR ($\oplus_2$)	Addition ternaire équilibrée ($\oplus_3$)
Espace clé	2^n	3^n (×1.58 plus grand)
Substrat	Électronique	Liaisons H de l'ADN (modèle)
Sécurité	Inconditionnelle	Inconditionnelle (Shannon 1949)

2. Définitions Mathématiques

2.1 Alphabet T_3

```
T3 = {−1, 0, +1}
```

2.2 Addition $\oplus_3$

```
a ⊕3 b = ((a + b + 1) mod 3) − 1,   ∀ a, b ∈ T3
```

$\oplus_3$	−1	0	+1
−1	+1	−1	0
0	−1	0	+1
+1	0	+1	−1

2.3 Soustraction $\ominus_3$

```
a ⊖3 b = a ⊕3 (−b),   −(−1)=+1,   −(0)=0,   −(+1)=−1
```

2.4 Espaces AT/CG

```
V_AT = T3²,   |V_AT| = 9   (2 liaisons H)
V_CG = T3³,   |V_CG| = 27  (3 liaisons H)
```

© 2026 kyriosMICA · Cyrille Egnon DAVOH · Tous droits réservés · direction@kyriosmica.com

3. Algorithme d'Encodage

3.1 Caractère → Vecteur T_3^n

ALGORITHME D'ENCODAGE

Entrée : caractère c, dimension n
Sortie : vecteur $v = (v_0, v_1, ..., v_{n-1}) \in T_3^n$

code ← ASCII(c)
Pour i de n−1 à 0 :
  v_i ← (code mod 3) − 1
  code ← ⌊ code / 3⌋

3.2 Exemple : 'A' = ASCII 65

Étape	code	trit
65 mod 3 = 2 → 2−1 = +1 ; code=21	21	+1
21 mod 3 = 0 → 0−1 = −1 ; code=7	7	−1
7 mod 3 = 1 → 1−1 = 0 ; code=2	2	0
2 mod 3 = 2 → 2−1 = +1 ; code=0	0	+1
0 (×3) → −1	0	−1,−1,−1

```
'A' (ASCII 65) → v = (−1,−1,−1,+1,0,−1,+1) ∈ T3⁷  [codon AT+CG+AT]
```

4. Chiffrement & Déchiffrement

4.1 Chiffrement

```
C[i][j] = M[i][j] ⊕3 K[i][j],   ∀ i ∈ blocs, ∀ j ∈ coordonnées
```

4.2 Déchiffrement

```
M[i][j] = C[i][j] ⊖3 K[i][j] = C[i][j] ⊕3 (−K[i][j])
```

4.3 Sécurité

```
I(M ; C) = 0   ⟷   H(M|C) = H(M)   (Shannon 1949)
```

AVERTISSEMENTS CRITIQUES

1. UNICITÉ : Si K est réutilisée, $C_1 \ominus_3 C_2 = M_1 \ominus_3 M_2$ — cryptanalyse possible.
2. QUALITÉ RNG : Sécurité nulle si générateur non cryptographiquement sûr.
3. TRANSMISSION CLÉ : Canal sécurisé indépendant obligatoire.

5. Format de Sérialisation

FORMAT DU MESSAGE CHIFFRÉ

$bloc_0$ | $bloc_1$ | ... | $bloc_{m-1}$

Chaque bloc : $trit_0,trit_1,...,trit_{n-1}$

Exemple (n=7, 2 caractères) :
-1,0,1,-1,0,0,1 | 0,-1,-1,1,0,-1,0

6. Vecteurs de Test

Char	ASCII	Codon	n	Vecteur attendu
A	65	AT+CG+AT	7	(−1,−1,−1,+1,0,−1,+1)
B	66	AT+CG+AT	7	(−1,−1,−1,+1,0,0,−1)
a	97	AT+CG+AT	7	(0,−1,0,+1,0,−1,0)

7. Implémentation Python

IMPLÉMENTATION COMPLÈTE

```
def t3add(a, b):
    s = a + b
    return s-3 if s>1 else (s+3 if s<-1 else s)

def t3sub(a, b): return t3add(a, -b)

def int_to_vec(n, length):
    v = []
    for _ in range(length):
        v.insert(0, (n % 3) - 1)
        n //= 3
    return v

def vec_to_int(v):
    return sum((t+1)*3**(len(v)-1-i) for i,t in enumerate(v))

def decrypt(cipher_blocs, key_blocs):
    result = []
    for i, cb in enumerate(cipher_blocs):
        kb = key_blocs[i % len(key_blocs)]
        plain = [t3sub(cb[j], kb[j]) for j in range(len(cb))]
```

```
        code = vec_to_int(plain)
        result.append(chr(code) if 32<=code<127 else '?')
    return ''.join(result)

# Parsing format sérialisé :
def parse(s):
    return [[int(t) for t in b.split(',')] for b in s.split('|')]
```

8. Instructions Défi

1. Lire ce guide et le White Paper (KM-DOC-004-2026)
2. Implémenter l'algorithme de déchiffrement
3. Valider sur les vecteurs de test (section 6)
4. Appliquer au message chiffré du défi
5. Soumettre à direction@kyriosmica.com — objet : [DÉFI QUDITS-36] Institution

SOUMISSION DU DÉFI

Email : direction@kyriosmica.com
Objet : [DÉFI QUDITS-36] Nom de votre institution
Délai : 12 mois à compter de la publication
Format : Message trouvé + méthode + code source

Decoding Life. Encoding the Future. — kyriosMICA, Bénin 2026

kyriosMICA

Decoding Life. Encoding the Future.

Bénin, Afrique de l'Ouest · kyriosmica.com · direction@kyriosmica.com

© 2026 Cyrille Egnon DAVOH — Tous droits réservés